

PCF DEVELOPMENT GUIDELINES

Folder Structure · Naming Conventions · CSS Styling

Power Platform · Model-Driven Apps
Version 1.0 · April 2026

1. Introduction

This document defines the development standards for Power Apps Component Framework (PCF) controls used in Model-Driven Apps. Consistent structure, naming, and styling practices reduce onboarding friction, improve maintainability, and ensure controls integrate cleanly with Dataverse and the Power Platform environment.

These guidelines cover three areas:

- Folder & file structure — how a PCF project is organized on disk
- Naming conventions — controls, files, properties, variables, and CSS identifiers
- CSS approach — when to use makeStyles (CSS-in-JS) vs. separate CSS files, and the rules for each

2. Folder Structure

2.1 Repository Layout

Each PCF control lives in its own folder inside a shared monorepo or solution repository. The top-level structure should be:

```

└─ solution-root/
  └─ controls/
    │   └─ MyControlName/
    │   │   └─ MyControlName/
    │   │   │   └─ __tests__/
    │   │   │   └─ components/
    │   │   │   └─ hooks/
    │   │   │   └─ styles/
    │   │   │   └─ utils/
    │   │   │   └─ index.ts
    │   │   │   └─ ControlManifest.Input.xml
    │   │   │   └─ package.json
    │   │   │   └─ tsconfig.json
    │   │   │   └─ .eslintrc.js
    │   └─ AnotherControl/
  └─ solution/
  └─ .gitignore
  └─ README.md
```

← PCF project folder (pac pcf init output)

← Unit tests

← Sub-components (if any)

← Custom React hooks

← CSS files (if not using makeStyles)

← Pure utility/helper functions

← PCF manifest entry point

← Power Platform solution files

NOTE

The outer folder (e.g. controls/MyControlName/) is the Git/repo folder. The inner folder with the same name is generated by pac pcf init and contains all source files.

2.2 PCF Project Folder Contents

Inside each PCF project folder, follow this internal layout:

```

└─ MyControlName/                                ← pac pcf init root
  └─ __tests__/
    │   └─ MyControlName.test.ts
    │   └─ utils.test.ts
    └─ components/                                ← Only when sub-components exist
      │   └─ MySubComponent.tsx
      │   └─ MySubComponent.styles.ts            ← co-located styles (makeStyles)
      └─ hooks/                                  ← Custom hooks only (no UI logic)
        │   └─ useMyHook.ts
        └─ styles/                              ← Only when using separate CSS files
          │   └─ MyControlName.module.css
          └─ utils/
            │   └─ formatters.ts
            │   └─ validators.ts
            └─ index.ts                          ← PCF class – thin wrapper only
            └─ MyControlName.tsx                 ← Root React component
            └─ ControlManifest.Input.xml
            └─ package.json
            └─ tsconfig.json
```

2.3 Folder Rules

Folder	Contains	Rule
__tests__/	All test files	Mirror source file names with .test.ts suffix
components/	Sub-components only	Create only when index.ts has 2+ visual sub-parts
hooks/	Custom React hooks	One hook per file; no JSX inside hooks
styles/	CSS / SCSS modules	Only present when not using makeStyles
utils/	Pure functions	No React imports allowed; fully unit-testable

RULE

The index.ts file must remain a thin PCF wrapper. It should contain only the class implementing `StandardControl<IInputs, IOutputs>` — no business logic, no styling, no JSX.

3. Naming Conventions

3.1 Control & Project Naming

The control name is the single most important identifier — it appears in the manifest, the namespace, the folder, and the import path. It must be:

- PascalCase
- Descriptive and domain-specific (e.g. LookupSearchField, not SearchControl)
- Free of generic words like Control, Component, Pcf, or Custom unless unavoidable
- Consistent between the folder name, manifest name, and React component name

✓ Do	✗ Don't
LookupSearchField	SearchControl
DateRangePickerField	DatePicker2
MultiSelectOptionSet	CustomMultiSelect
RichTextInputField	RichTextPCF
StatusBadgeIndicator	statusBadge

3.2 Namespace

The namespace in ControlManifest.Input.xml must follow reverse-domain notation using your organisation prefix:

```
<!-- ControlManifest.Input.xml -->
<control namespace="CompanyName.Controls"
  constructor="LookupSearchField"
  version="1.0.0"
  display-name-key="LookupSearchField_DisplayName"
  description-key="LookupSearchField_Desc"
  control-type="standard">
```

RULE

Always use two segments: OrganisationName.Controls. Never use generic namespaces like PCF.Controls or Microsoft.Controls.

3.3 File Naming

File Type	Convention	Example
PCF entry point	PascalCase + .ts	LookupSearchField/index.ts

File Type	Convention	Example
Root component	PascalCase + .tsx	LookupSearchField.tsx
Sub-component	PascalCase + .tsx	SuggestionDropdown.tsx
makeStyles file	PascalCase + .styles.ts	LookupSearchField.styles.ts
CSS module	PascalCase + .module.css	LookupSearchField.module.css
Hook file	camelCase prefixed use + .ts	useDebounceSearch.ts
Utility file	camelCase noun + .ts	formatters.ts / validators.ts
Test file	Mirror source + .test.ts	LookupSearchField.test.ts
Manifest	Fixed name (pac standard)	ControlManifest.Input.xml

3.4 Manifest Property Naming

Input and output properties defined in the manifest must follow these conventions:

- PascalCase for property names
- Use clear, noun-based names that describe what the property represents — not how it behaves
- Boolean properties should start with Is, Has, Allow, or Show
- Event properties should start with On

☑ Do	✗ Don't
SelectedValue	value
IsDisabled	disabled
AllowMultiSelect	multiSelect
OnValueChange	change
PlaceholderText	placeholder_text
MaxSelectionCount	maxCount

3.5 TypeScript Naming

Inside TypeScript and TSX files, apply the following conventions consistently:

Identifier	Convention	Example
Interface	PascalCase, I prefix	ILookupItem, IControlProps
Type alias	PascalCase	SelectionMode, DropdownOption

Identifier	Convention	Example
Enum	PascalCase (name + value)	enum SortDirection { Asc, Desc }
Constant (module-level)	SCREAMING_SNAKE_CASE	MAX_SUGGESTIONS
Variable / function param	camelCase	selectedItems, handleChange
Private class member	camelCase (no underscore)	this.notifyOutputChanged
React component	PascalCase	SuggestionDropdown
Custom hook	camelCase, use prefix	useDebounceSearch
Utility function	camelCase verb	formatDisplayName()

3.6 CSS / Class Naming

CSS class naming conventions differ depending on whether you use `makeStyles` or separate CSS files — see Section 4 for full details. The shared rules across both approaches are:

- Use camelCase for `makeStyles` keys
- Use kebab-case for CSS module class names
- Never use generic names like `wrapper`, `container`, `inner`, `box` without a domain qualifier
- Never use presentational names (`blueBorder`, `redText`) — use semantic names (`errorState`, `activeItem`)

4. CSS Approach

Both `makeStyles` (CSS-in-JS) and separate CSS module files are permitted. The choice is not arbitrary — each approach has a defined home. Mixing them without reason inside a single control is not allowed.

4.1 Decision Guide

Use <code>makeStyles</code> when...	Use CSS modules when...
Styles depend on runtime props or state	Styles are fully static
Using Fluent UI components throughout the control	Minimal or no Fluent UI usage
You need theming token integration	Styles are simple and self-contained
Co-location with the component matters	Designers provide standalone CSS/SCSS
Dynamic class composition is required	Control needs to be styled externally

RULE

If the same control has dynamic, state-driven styles and a handful of static utility classes, prefer `makeStyles` throughout for consistency. Do not split the two approaches within one component.

4.2 `makeStyles` Rules

File placement

When using `makeStyles`, styles must live in a co-located `.styles.ts` file — never inline inside the component file.

```
// ☒ Co-located style file
// LookupSearchField.styles.ts

import { makeStyles, shorthands, tokens } from '@fluentui/react-components';

export const useLookupStyles = makeStyles({
  root: {
    display: 'flex',
    flexDirection: 'column',
    ...shorthands.gap('8px'),
  },
  inputWrapper: {
    position: 'relative',
    width: '100%',
  },
});
```



```

dropdownList: {
  backgroundColor: tokens.colorNeutralBackground1,
  boxShadow: tokens.shadow8,
  borderRadius: tokens.borderRadiusMedium,
},
dropdownItem: {
  ':hover': {
    backgroundColor: tokens.colorNeutralBackground1Hover,
  },
},
activeItem: {
  backgroundColor: tokens.colorBrandBackground2,
  fontWeight: tokens.fontWeightSemibold,
},
errorState: {
  ...shorthands.borderColor(tokens.colorPaletteRedBorder2),
},
});

```

Usage in component

```

// LookupSearchField.tsx
import { useLookupStyles } from './LookupSearchField.styles';

export const LookupSearchField: React.FC<IProps> = ({ isError }) => {
  const styles = useLookupStyles();

  return (
    <div className={styles.root}>
      <div className={mergeClasses(styles.inputWrapper, isError &&
styles.errorState)}>
        ...
      </div>
    </div>
  );
};

```

makeStyles key naming rules

- Keys are camelCase semantic names: root, inputWrapper, dropdownItem, activeItem, errorState
- Always name the outermost element root
- Never use element names as keys (div, span, input) — use role/purpose names
- State variants are suffixes: activeItem, disabledItem, errorState, loadingState
- Always use Fluent UI tokens (tokens.*) for colours, spacing, shadows, and typography — never hardcode hex values or pixel sizes

4.3 CSS Module Rules

File placement

CSS modules go in the `styles/` folder and use the `.module.css` extension.

```
/* styles/LookupSearchField.module.css */

.root {
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.inputWrapper {
  position: relative;
  width: 100%;
}

.dropdownList {
  background: var(--pcf-surface, #ffffff);
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.12);
  border-radius: 4px;
}

.dropdownItem:hover {
  background: var(--pcf-hover-bg, #f5f5f5);
}

.activeItem {
  background: var(--pcf-selected-bg, #e8f0fe);
  font-weight: 600;
}

.errorState {
  border-color: var(--pcf-error, #a4262c);
}
```

Usage in component

```
// LookupSearchField.tsx
import styles from '../styles/LookupSearchField.module.css';

export const LookupSearchField: React.FC<IProps> = ({ isError }) => {
  const wrapperClass = [
```

```
    styles.inputWrapper,  
    isError ? styles.errorState : '',  
  ].filter(Boolean).join(' ');  
  
  return (  
    <div className={styles.root}>  
      <div className={wrapperClass}>  
        ...  
      </div>  
    </div>  
  );  
};
```

CSS module naming rules

- Class names are camelCase (CSS modules map them as object keys): `.inputWrapper`, `.activeItem`
- The outermost element always has the class `.root`
- Use CSS custom properties (`--pcf-*`) for values that may vary by theme or app, rather than hardcoded values
- Never use ID selectors or element selectors (`div`, `span`) in module files
- Never use `!important` — restructure specificity instead

4.4 Shared CSS Rules (Both Approaches)

Rule	makeStyles	CSS Module
Outermost element class	<code>styles.root</code>	<code>styles.root</code>
State variants	Suffix: <code>errorState</code> , <code>activeItem</code>	Suffix: <code>.errorState</code> , <code>.activeItem</code>
Color values	<code>tokens.*</code> from Fluent UI	<code>--pcf-*</code> custom properties
Spacing values	<code>tokens.*</code> or shorthands	4px / 8px grid — no arbitrary values
Never use	Hardcoded hex/px	ID selectors, <code>!important</code>
Dynamic styles	<code>mergeClasses()</code>	<code>Array.join</code> or <code>classnames</code> lib
Global overrides	Not allowed	Not allowed (use <code>:global()</code> as last resort only)

4.5 Spacing & Sizing System

Whether using `makeStyles` or CSS modules, all spacing and sizing values must follow an 8px base grid:

- 4px — micro spacing (icon gaps, input padding adjustments)
- 8px — default gap between elements
- 16px — section spacing, card padding
- 24px — component-level margin
- 32px+ — layout-level spacing

RULE

Never use arbitrary pixel values like 7px, 11px, or 22px. If a design calls for an off-grid value, round to the nearest grid step and flag with the designer.

5. Quick Reference

5.1 New Control Checklist

Use this checklist when creating a new PCF control from scratch:

- Run `pac pcf init` with PascalCase name matching the domain feature
- Create the `controls/<ControlName>/` repo folder and move the generated project inside
- Set namespace to `CompanyName.Controls` in the manifest
- Create `__tests__`, `utils` folders immediately
- Create `components` and `hooks` only when needed (not by default)
- Decide on `makeStyles` vs. CSS modules before writing any styles — see Section 4.1
- Keep `index.ts` as a pure PCF wrapper — move all logic to the `.tsx` file
- Name all manifest properties in PascalCase with descriptive nouns
- Name all CSS classes semantically — no presentational or element-based names

5.2 Naming at a Glance

What	Convention	Example
Control name	PascalCase	LookupSearchField
Namespace	OrgName.Controls	Contoso.Controls
Manifest property	PascalCase noun	SelectedValue, IsDisabled
TypeScript interface	I + PascalCase	ILookupItem
Module constant	SCREAMING_SNAKE	MAX_SUGGESTIONS
Hook file & export	use + PascalCase	useDebounceSearch
<code>makeStyles</code> key	camelCase semantic	activeItem, errorState
CSS module class	camelCase semantic	.activeItem, .errorState
CSS custom property	--pcf- prefix	--pcf-surface, --pcf-error

5.3 CSS Approach Decision Tree

```

Does the control use Fluent UI components?
├── Yes  → Does styling depend on props/state/theme tokens?
│   ├── Yes → Use makeStyles
│   └── No  → Use makeStyles (stay consistent with Fluent UI)
└── No   → Are styles entirely static?

```

- |— Yes → Use CSS modules
- |— No → Use makeStyles (dynamic styles are easier to manage)

⚠ Never mix makeStyles and CSS modules within the same component file.